AMME4710 Computer Vision and Image Processing

THE UNIVERSITY OF SYDNEY

Assignment 1 (20%), 2026

- This assignment is due on Sunday 30th August 2026 at 11:59pm, to be submitted using the Turnitin links on the course's Canvas website (see instructions below).
- This assignment contributes 20% to your final mark.
- Late assignments will be deducted 5% (5 marks out of a possible 100) for each day late, starting on the day the assignment is due and including weekends. Assignments submitted more than 10 days late will receive zero.
- Any special consideration requires you to go through the Special Consideration form process through Sydney Student.
- Incidences of academic dishonesty or plagiarism will be referred to the academic honesty coordinator and could result in a zero mark being awarded for this assignment, or automatic failure of the entire subject.
- This assignment should take the average student 12 hours to complete.

Objectives

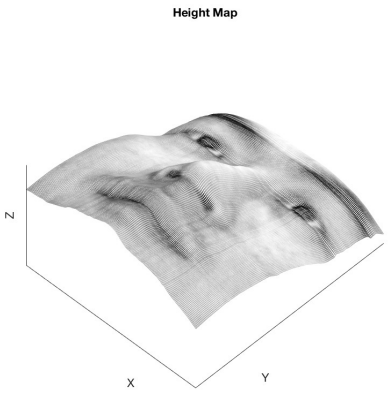
- In this assignment you will develop computer vision algorithms for solving problems involving light, shading and colour.
- You will develop an algorithm for recovering the 3D shape of a face using photometric stereo and develop image processing techniques to determine the state of a connect-four game board using colour and morphology information.
- For each of these problems, you will develop python code that implements your algorithms and write a report examining existing approaches to your problem, detailing your design process, and presenting and examining your results using one or more different image datasets.

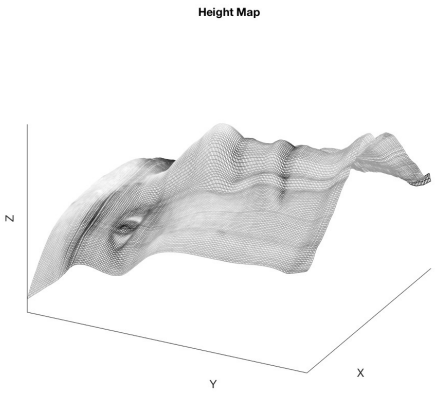
Submission

- The assignment is due by 30th August 2026 11:59pm.
- You will provide one report file for the assignment. The report will include the Sections "Question 1", "Question 2" and an appendix with subsections for each question. Each of the first two sections will contain the subsections "Introduction", "Methodology" and "Results and Discussion".
- For each question you will submit **working** python code. This can either be in a Jupyter notebook or standalone Python code (for each question make sure there is a script file named "mainQ**N**.py" where **N** is the question number). This script file will be run by the tutors during marking and should run all of your code and produce all of your plots for the questions.
- There are TWO submission portals for this assignment, one for the report and one for your python code, both via Turnitin on the course Canvas site:
    - The report should be submitted to the "**Report Submission**" portal

    - Your python code is to be zipped into a single file (i.e. "**SID**_Assignment1.zip" where **SID** is you student number) and submitted via the "**Code Submission**" portal.
    - You will need to submit both your report AND python code to receive marks for the assignment.
- All python code must be commented so that the code can be read and understood without the aid of the report.
- If your code does not work, you will receive **0** marks for the code section of the marking unless you provide an explanation in the report as to why your code didn't work.
- Acknowledging use of AI Tools: If you use generative AI tools to assist in working on your assignment (for example LLMs such as ChatGPT, Claude, Microsoft Co-pilot etc.), you are required to acknowledge it: this includes acknowledging any tools that use generative AI, such as translation tools, paraphrasing tools or referencing tools:
    - Include a statement at the end of your report acknowledging any tools you have used, and how you have used them to assist
    - Allowable uses include suggestions for paraphrasing and use of language in reports, feedback on report structure and presentation, suggestions for code in your implementation of experiments etc.
    - Remember that you are ultimately responsible for the work you present and submit: don't use AI tools to subvert your own learning
    - For more details on responsible AI tool use in assessments, see: https://www.sydney.edu.au/students/responsible-ai-use.html

Question 1: Photometric Stereo (50 %)





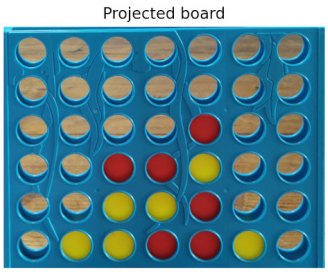
- This question is an extension of the photometric stereo tutorial activity performed during week 2. You should refer to the week 2 tutorial sheet on instructions for loading and viewing the datasets described here, and you should have successfully completed tutorial activities 1 and 2 from week 2 before attempting this question.
- Using your recovered surface normals for each of the face datasets, develop an algorithm for recovering a 3D height image of each subject's face using numerical integration across the image in the x and y directions.
    - Your approach should set the height of the top left pixel to be zero as a starting point for each path integral.
    - You should implement three different integration strategies:
        - (a) integrate in the horizontal direction along the top row first, then down each column
        - (b) integrate in the vertical direction first along the first column, then across each row
        - (c) the average of height computer by (a) and (b)
    - Your integrations will essentially be cumulative sums of the gradient values treating dx and dy to be a single pixel distance (i.e. equal to 1)
    - See the numpy function "*cumsum*" for calculating the cumulative sum in either a horizontal or vertical direction.
- Use the provided script code in "plot_face_3d.py" available in the assignment canvas link to display a 3D rendered image of the face (you will also need to provide this function with a recovered height map and albedo image of the face, as performed in the week 2 tutorial activity)
- Once you have working reconstructions for each of the faces in the datasets B01, B02, B05 and B07, try to improve the accuracy of your approach by developing and outlier detection and rejection system in your code for recovering the surface normals:
    - For each pixel (x,y), compute the residual between the measured pixel brightness and the pixel brightness predicted by your values for albedo, surface normal and lighting direction (see Week 2 tutorial, activity 1)
    - Produce a plot, for each pixel in each face image (displayed as a montage) that highlights image regions for which the residual magnitude is greater than two times the standard deviation of residuals for that pixel.

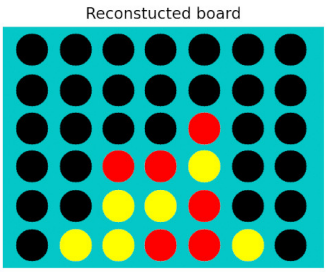
    - Try to re-implement your surface normal computing algorithm and/or 3D height computing algorithm to not use pixel brightness data outside this threshold
- Your python code may use packages for opencv, numpy, matplotlib, and any other built-in python modules. You may not use other external code or libraries: ask the teaching team if you have any other python packages you are interested in using.
- In your report, under "Question 1", detail your algorithm and present and compare results. This section of the report (Question 1) should have the following subsections:
    - **Introduction:** discuss the basic principles behind how photometric stereo works, and from your own research describe some of the existing approaches and applications of photometric stereo in the research literature
    - **Methodology:** Describe how your algorithms for 3D reconstruction and outlier detection and rejection were developed and implemented.
    - **Results and Discussion:** Use plots from your python code to demonstrate results on each of the face datasets provided. Include key figures and/or tables in the main body of your report and provide additional plots in an appendix at the end of your report and reference them in the body of your report.
    - This section of the report has a **4-page limit** (not including the appendix).

Question 1 Marking Scheme

Code Implementation (20/50)	Code doesn't work	**0** Marks awarded for code
	Code works, is readable and commented and the implementation is correct: Have the 3D heights been properly reconstructed? Has an outlier detection and reject mechanism been implemented into a surface normal/albedo computing implementation?	**/20** Marks
Report (30/50)	Introduction: discuss the basic principles behind how photometric stereo works, and from your own research describe some of the existing approaches and applications of photometric stereo in the research literature.	**/5** Marks
	Methodology: Have the methods implemented been clearly explained in the report?	**/10** Marks
	Discussion and Analysis: How has the algorithm performed over the different face datasets? Where obvious errors are present, what is the reasoning for this in terms of the assumptions that go into the method? What effect has the outlier rejection system had on the results?	**/15** Marks

Question 2: Connect-four game board state determination (50%)





Overview:
In this question you will develop and evaluate your own algorithm for determining the state of a connect-four game board from an image of the board.

The connect-four board, with its distinct light blue colour (see above), contains a seven-by-six grid of playable cells that players take turns at placing their piece in, using the slot at the top of the board. One player uses yellow tokens/pieces, whereas the other player uses red. At any point in the game, each of the 42 cells on the board can be in one of three states:

- Empty/unplayed (0)
- Occupied by a yellow token (1)
- Occupied by a red token (2)

The game board state can be represented as a 6-by-7 matrix, with each cell taking on one of the values (0, 1 or 2) as described above.

Instructions:

- On the assignment Canvas page, you will find a link to a dataset of images ("**connect_four_images_A1.zip**") of the same connect-four game board, but with varying states of play and taken from various perspectives.
- You should develop an image processing algorithm in python which considers a single input image at a time, and outputs the estimated board state in the matrix form specified above.
- Your algorithm should utilise/exploit the colour and shape properties of the board and tokens via image processing techniques in colour thresholding and basic approaches to analysing image morphology to determine the board state.
- Your algorithm should be implemented in a python function which inputs a numpy array containing the colour image data (i.e. (3472-by-2598-by-3) for the images in the dataset) and outputs a 6-by-7 numpy array, with values of (0,1,2) representing the estimated board state.
- In order to evaluate the performance of your detection/tracking algorithm, you will also be provided with validation data associated with each image. Download **"assign1Q2_validationdata.zip"** which contains two python pickle files which contain "ground-truth" information for each image:

    - **“board_corners.pkl”:** contains the four (x,y) coordinates for the corners of the game board in each image, in the order (upper left, upper right, lower left, lower right).
    - **“board_states.pkl”:** contains a 6-by-7 ground truth board state for each image
- You should also write a python script (i.e. MainQ2.py) that loads images, calls your implemented function and calculates the accuracy from your results (see below) using the “ground truth” validation information. When evaluating your algorithm’s accuracy, you should produce numbers for:
    - Per-image/game board results:
        - **Board Accuracy:** percentage of the 42 cells which have been correctly identified for each board
    - **Average Board Accuracy:** computed over all of the example images
    - **Overall Accuracy:** percentage of images with every cell of the board correctly determined
    - You should also consider additional metrics, such as accuracy in identifying the board location in the image, if your approaches uses this as an intermediate processing step.
- Your python code may use packages for opencv, numpy, matplotlib, and any other built-in python modules. You may not use other external code or libraries: ask the teaching team if you have any other python packages you are interested in using.

Suggested Approach:

- Since the board and tokens are a distinct colour, you should attempt to threshold their locations using colour thresholding techniques explored during Week 2.
- Idea 1:
    - Try to determine the position of the board including the locations of the four corners of the board. Morphology and contour-finding algorithms might help during this step.
    - Once you find the board corners, you can apply a perspective transformation using the functions “cv2.getPerspectiveTransform” (and possibly “cv2.warpPerspective”) to normalise the perspective of the board to allow you to locate the coordinates of each cell, from which you can ascertain their colour
- Idea 2:
    - Try to determine the positions of the game board cells directly in the image by detecting tokens and/or empty spaces on the board. Morphology and contour-finding algorithms might help during this step.

Report:

- In your report, under “Question 2”, detail how your algorithm was developed, the design decisions made and present validation and performance results. This section of the report (Question 2) should have the following subsections:
    - **Introduction:** From your own research describe some of the existing approaches to colour-based object tracking and detection and describe some of the difficulties faced in tracking and recognising objects based on colour.

- **Methodology:** Detail the development steps you went through to design your approach to the problem, discuss the design decisions made and explain the final approaches taken.
- **Results and Discussion:** Show examples of labelled images and detail your validation results for your final algorithms. Use plots from your python code to examine and explain images for which your algorithm works well on and images that it performed worst on, and why this might be the case. You will also want to display and illustrate any results of the intermediate steps in your processing, e.g. thresholding operations, board location detection, token detection etc.
- This section of the report has a **5-page limit** (not including the appendix).

Question 2 Marking Scheme

Code Implementation (20/50)	Code doesn't work	**0** Marks awarded for code
	Code works, is readable and commented. Are the implemented algorithms potentially robust to various types of images and have they been implemented in a sensible manner with a reasonable level of computational efficiency?	**/20** Marks
Report (30/50)	Introduction: discuss some existing approaches to colour-based object tracking and describe some of the difficulties faced in tracking and recognising objects based on colour.	**/5** Marks
	Methodology: Have the methods implemented been clearly explained in the report? Has reasoning been given for design decisions made?	**/15** Marks
	Discussion and Analysis: Has the validation analysis been performed correctly and clearly explained? How has the algorithm performed over the different images? Where obvious errors are present, what is the reasoning for this in terms of how the methods work? How could the approach be improved upon, and what additional considerations might be important for the task if further developing it to work on a wider variety of images in different environments?	**/10** Marks